Prototype Learning for Quantum Neural Networks

Bodo Rosenhahn[1], Christoph Hirche[1]
1) Institute for Information Processing (tnt/L3S),
Leibniz Universität Hannover, Germany
Corresponding Authors: {rosenhahn,hirche}@tnt.uni-hannover.de
(Dated: August 20, 2026)

This work proposes a quantum prototype perceptron architecture that fundamentally enhances the representational capacity and generalization ability of quantum neural networks. Our approach adapts the principles of prototype learning, prominently used in classical interpretable machine learning, by introducing learnable reference patterns (prototypes) directly within quantum circuits. In addition, we present a quantum optimization procedure that exploits quantum superposition to encode the entire training dataset as a quantum state, allowing for global, distribution-level adjustment of network parameters. The model parameters are optimized by minimizing the Kullback-Leibler divergence between the target and model output distributions, leading to significantly accelerated convergence and reduced classical memory requirements. Empirical evaluations across regression and classification tasks demonstrate that quantum prototype networks substantially decrease susceptibility to overfitting and achieve superior generalization compared to standard approaches.

I. INTRODUCTION

Artificial neural networks based on a classical perceptron model is the foundation of modern machine learning. It enables to solve classification, regression, and representation tasks across diverse domains. Former works proposed a probabilistic perceptron, especially in its quantum realization, that has shown notable flexibility and efficiency—particularly for binary data encoded on quantum devices [26]. But its classic formulation is inherently limited by the nature of its activation mechanism. In the standard perceptron, only input values set to one positively contribute to the activation probability, resulting in an asymmetric and sometimes suboptimal representation, especially in scenarios with high redundancy or correlated features. Consequently, such models are frequently prone to overfitting, which can lead to instability and poor generalization on unseen data.

Prototype learning [28] offers a well recognized paradigm shift: Rather than encoding knowledge primarily as adjustable weights, prototype-based models leverage learnable reference patterns, called prototypes that perform comparisons against input data. Specifically, by realizing a comparison operation akin to the XOR logical function between input vectors and prototypes, both zero and one entries can contribute meaningfully to the perceptron activation. This bidirectional sensitivity enables a richer and more interpretable feature representation, which also motivated the principles underlying modern attention mechanisms in transformer architectures [30].

In this work, we introduce a quantum prototype perceptron architecture, which extends previous quantum neural network models by integrating efficient, prototype-based comparison directly into the gate-based quantum circuit framework. We further present a significant advancement in the optimization strategy. Whereas previous training schemes are iterative and operate on individual samples or small batches, often involving repeated state preparations and costly measurements, we exploit the unique properties of quantum superposition. By encoding the entire training set simultaneously as a quantum state, we enable a global, distribution-level optimization of the model parameters, minimizing the Kullback-Leibler divergence between the target and model output distributions in a single operation. This not only leads to efficient convergence but it also reduces the need for explicit data storage, as the training set is embedded implicitly within the quantum state.

Our initial experiments indicate that quantum prototype learning is substantially more robust to overfitting than classical and conventional quantum perceptron models. The prototype approach yields stable training and

superior generalization performance, as evidenced in benchmark regression and classification tasks. Thus, quantum prototype learning offers a viable route towards scalable and interpretable quantum machine learning models, combining the strengths of prototype methods with the unprecedented parallelism and representational power of quantum computing.

In summary, our main contributions are:

1. The formulation of a novel quantum prototype perceptron model that enables both 0- and 1-valued input features to actively participate in the activation computation, mirroring an XOR-type comparison with learnable reference vectors.

2. The development of a quantum optimization procedure based on superposition encoding, allowing training over the entire data distribution simultaneously and achieving significant efficiency gains in comparison to conventional, samplewise optimization.

3. Empirical evidence showing that prototype quantum models exhibit increased stability and are less susceptible to overfitting than standard perceptron-based quantum networks.

By bridging prototype-based representation learning and quantum computation, our work establishes a new direction for the design and training of interpretable, efficient, and robust quantum neural networks.

The remainder of this article is organized as follows. Section 2 provides a concise overview of the fundamental concepts underpinning this work, including classical and quantum perceptron models and optimization strategies relevant for quantum neural networks. In Section 3, we introduce the proposed quantum prototype perceptron architecture in detail, describing both the circuit-level design and its operational principles. Afterwards we present our approach for global quantum optimization via superposition-based training, outlining the formalism for encoding entire datasets and performing distribution-level loss minimization. This part is followed by an empirical evaluation of the proposed model across benchmark tasks in classification and regression. We finally present a Flappy-Birds demonstrator, played by our prototype quantum neural network. The paper finishes with a brief summary and outlook.

II. FUNDAMENTALS

A. Neural Networks

In 1958, Frank Rosenblatt published his perceptron model which is based on a scalar product, followed by a step function, see Rosenblatt [25]. It can be summarized as

$$\sigma_j = \begin{cases} 1 : \sum_i \omega_{ij} x_i + b > 0 \\ 0 : else \end{cases} \tag{1}$$

The bias value b corresponds to the decision threshold and ω_{ij} are learnable parameters. A combination of such perceptrons in a directed acyclic graph leads to a classical (e.g. fully connected) neural network.

Since the amount of variables to optimize increase drastically with increasing input dimensions, layers and perceptrons, which can lead easily to overfitting, several models have been proposed to reduce the amount of variables. Two famous examples are local-receptive fields and convolutions, which are commonly used in image analysis. The so-called attention mechanism is a central innovation that underpins the nowadays often used transformer architecture. It enables a model to dynamically focus on relevant parts of input data when making predictions, regardless of the input sequence's length or position. For a given a sequence of input vectors (for example, word embeddings in natural language processing), the attention mechanism computes, for each position in the sequence, a weighted sum of all the input vectors. The weights—attention scores—are computed dynamically, depending on the relationship between elements of the input. Most common is the scaled dot-product attention. For a set of input embeddings, three distinct representations are computed via learned linear projections, called Queries (Q), Keys (K) and Values (V). For each query, the attention score with every key is calculated using a dot product, normalized (by the square root of the key

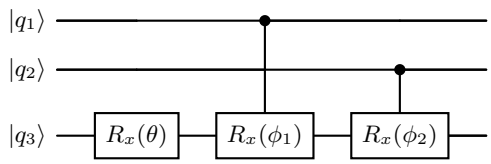


FIG. 1. Realization of a perceptron as proposed in [26]. The input qubits $|q_1\rangle$ and $|q_2\rangle$ modify the probability of activating $|q_3\rangle$.

dimension), and then passed through a softmax function to create a probability distribution. Let $\mathcal{D}$ be a database with corresponding key and values, $\mathcal{D} = \{(\mathbf{k}_1, \mathbf{v}_1), \ldots, (\mathbf{k}_m, \mathbf{v}_m)\}$. Let $\mathbf{q}$ be a query. Then attention over $\mathcal{D}$ can be computed as as

$$f(\mathbf{q}, \mathcal{D}) = \sum_{i=1}^{m} \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i \tag{2}$$

where $\alpha(\mathbf{q}, \mathbf{k}_i)$ are scalar attention weights.

The attention mechanism is used in transformers [30] as so-called multi-head attention. This is commonly used in Large Language Models [4, 19], Vision Language Models [31] and general foundation models [10, 13, 17, 18].

B. Quantum Neural Networks

Quantum Neural Networks (QNNs) represent a hybrid mathematical framework integrating principles of quantum computing with artificial neural networks. Mirroring biological architecture, classical neural networks are based on perceptron which rely on structurally straightforward elements, typically a scalar product combined with a non-linear activation function. Differentiable functions permit efficient optimization via gradient descent. When assembled into vast networked hierarchy, these models serve as powerful universal function approximators [11]. Seeking to translate these capabilities into the quantum domain, various architectures leverage parameterized quantum circuits [5] to replicate core deep learning components. Notable examples include quantum variants of convolutional layers [3], recurrent neural networks designed for sequential or time-series data [1], and neuromorphic spiking networks [16]. Alternatively, the approach introduced in [2] maps individual input features and individual nodes to distinct qubits, utilizing localized unitary operations to execute both data processing and neuronal activation.

Although these paradigms demonstrate significant theoretical promise, they introduce fundamental architectural deviations from standard deep learning pipelines. For instance, the sequential application of unitary matrices in [2] imposes a non-commutative execution chain; this is in contrast to classical layer topologies where individual neurons are processed fully in parallel and independently of one another. Furthermore, alternative implementations frequently demand additional infrastructural overhead, such as auxiliary ancilla qubits or non-deterministic repeat-until-success circuit protocols [5]. For this reason, the former work XY present a more minimalistic way to realize neurons. Figure 1 summarizes the core principle: Two input qubits are connected to an output qubit by using controlled rotation (CRX) gates. A plain rotation realizes the bias. This concept leads to a probabilistic activation of the third qubit, dependent on the input activations.

C. Optimization of network weights

Inspired by the former work XY, the parameters of the quantum prototype neural network are optimized using stochastic approximation methods. The Kiefer-Wolfowitz algorithm [12], which is closely connected to the Robbins-Monro scheme [24]. The method estimates the gradient of an objective function from noisy function evaluations. For a sufficiently smooth objective function $M(x)$, the iterative update is given by

$$x_{n+1} = x_n + a_n \left(\frac{M(x_n + c_n) - M(x_n - c_n)}{2c_n} \right)$$

where $M(x_n + c_n)$ and $M(x_n - c_n)$ are independently computed. Thus, the gradient of $M(x)$ is approximated by a central difference method. A damping factor a_n as step size controls the shifts the solution towards an optimum.

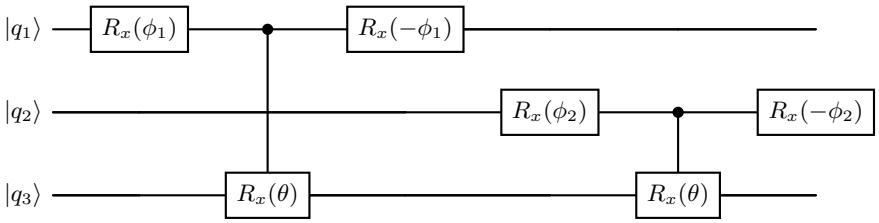


FIG. 2. Our realization of a prototype sensitive perceptron. rotations before the CR-rotations modify the likelihood for increasing an activation on $|q_3\rangle$. This allows to capture also 0-entries as valuable information. Figure 3 shows a tiny example which leads to the highest activation for a [1 0] input prototype.

To further improve the exploration of the optimization landscape, we employ a simulated annealing (SA) scheme [14]. A typical function for P takes the form

$$P(e(x_t), e(\hat{x}_t), T) \propto \exp\left(-\frac{e(\hat{x}_t) - e(x_t)}{kT}\right), \tag{3}$$

with k a *damping factor* $k > 0$. At iteration t, a candidate parameter vector $\hat{x}_t$ is sampled in the neighborhood of the current vector x_t, for example using a stochastic perturbation. The candidate is accepted according to an acceptance probability that depends on the objective function $e(x)$, which corresponds to the optimization loss in this work. Candidate solutions that reduce the loss are therefore always accepted. Solutions with an increased loss may also be accepted, with a probability that decreases with both the loss increase and the temperature. As T_t decreases according to an annealing schedule, the optimization gradually changes from exploratory search to local refinement. In contrast to strict gradient descent, this mechanism can help to escape local minima.

As shown in XY, common gradient-based optimization is also possible, but due to the risk of frequently getting stuck in local minima, our implementation is solely based on stochastic optimization.

III. QUANTUM PROTOTYPE LEARNING

This section introduces our approach for learning prototype models on a quantum architecture. The section starts with a more general introduction to prototype learning, then it continues with our prototype perceptron model. Afterwards our learning strategy under superposition is introduced and its impact demonstrated on several experiments.

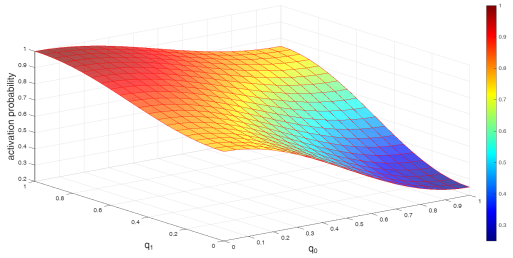


FIG. 3. activation probability of the prototype $|01\rangle$: If $|q_0\rangle$ is in 0 and $|q_1\rangle$ is in 1, the probability of activation is maximized. If only one condition is met, the activation is reduced and if the pattern does not match, the activation is zero

A. Prototype Learning

Prototype learning is a machine learning paradigm in which the model learns to represent classes, clusters, or relevant features of the input data through one or more representative *prototype* vectors [29]. Instead of focusing purely on optimizing numerical weights (as in standard neural networks), prototype learning algorithms explicitly form these prototypes from the data, and classify or interpret new inputs by comparing them to the learned prototypes. Each prototype can be thought of as a learned reference point in feature space that

represents a cluster, class, or pattern in the data. In several cases, a prototype can be an actual data point or an artificial vector in the feature space. New input samples are classified or regressed by measuring their similarity (often via a distance metric like Euclidean distance or cosine similarity) to all available prototypes. The model's prediction is then based on which prototypes are closest, or on a weighted vote of their similarities. Equation 2 for the attention mechanism is one option to realize such a concept on a neural network.

Prototype-based models tend to be less prone to overfitting compared to models with large numbers of weights, since the learning task involves summarizing groups of data rather than memorizing specific inputs. With regards to existing methods, the k-Nearest Neighbor (k-NN) method can be seen as the simplest, yet still very efficient form, where actual training samples are used to determine prototypes [6]. In [27] prototypes are generated during optimization in a feature space to represent different classes. This is then called Learning Vector Quantization (LVQ). An extension to this is Relevance Learning Vector Quantization. Here, prototypical networks are used in few-shot and meta-learning. Prototypes are then centroids of class clusters in an embedding space [9].

B. Prototype perceptron model

Coming back to Figure 1 which summarizes the core principle of a classical stochastic perceptron: Two input qubits are connected to an output qubit by using controlled rotation (CRX) gates and a plain rotation realizes the bias. As this model only changes it probability for the controlled gates, it does not learn any real prototypes, as also a qubit with a 0-value can carry important information. This is one of the reasons why we propose to change the architecture towards a prototype variant. Figure shows 2 the core idea for the realization of a prototype sensitive perceptron: Rotations before the CR-rotations modify the likelihood for increasing/decreasing an activation on $|q_3\rangle$. This allows to capture also 0-entries as valuable information. Figure 3 shows a tiny example which leads to the highest activation for a [1 0] input prototype. Thus, only if qubit $|q_1\rangle = 1$ and $|q_2\rangle = 0$ the qubit $|q_3\rangle$ is set to 1. But if $|q_1\rangle = 1$ and $|q_2\rangle = 1$ it then leads to a decrease of the probability to activate the qubit $|q_3\rangle$. Please note, that this model itself does require an increase of optimization parameters, as the former bias is shifted to the amount of activation when a rotation is triggered and the parameters of the controlled rotations are shifted to the rotations before and after the CR-gate.

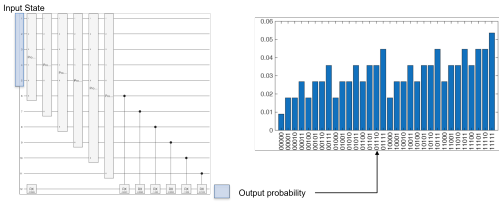


FIG. 4. Quantum Prototype Learning architecture: The task is bit-counting of a 5-dimensional bit string. The 5 bits are encoded as input (denoted as shaded area). Prototype layers connect to 6 hidden neurons which are then forwarded to one output neuron. The output neuron gives are probability of measurement, which is an amplitude encoding of the output. Strings with many 1-bits result in a higher activation probability and bitstrings with many 0 entries result in a lower probability.

C. Learning prototypes under superposition

Figure 4 summarizes the basic pipeline on a small example: The task is bit-counting of a 5-dimensional bit string. The 5 bits are encoded as input (denoted as shaded area). Prototype layers as defined in Figure 2 are connected to 6 hidden neurons. The outcome is then forwarded to one output neuron. The output neuron gives a probability of measurement. It yields an amplitude encoding of the desired bit count. Strings with many 1-bits result in a higher activation probability and bit strings with many 0 entries result in a lower probability.

The work [26] proposes an iterative scheme for optimizing the neural network weights. A

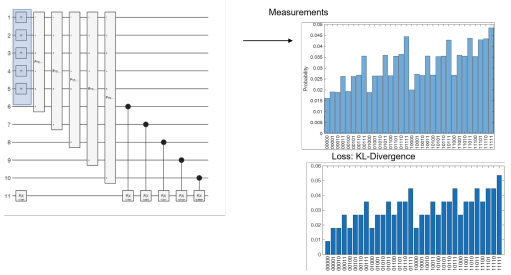


FIG. 5. Efficient optimization: Hadamard gates are used to encode all possible states in superposition. The measured outcome distrbitution can compared to the target distribution using e.g. a KL-divergence score. This allows for a global optimization of the network weights.

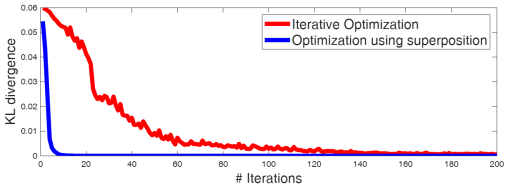


FIG. 6. Efficient optimization: Optimizing examples iteratively leads to a slow convergence behavior (red), whereas training in superposition (blue) leads to a drastically faster convergence.

sample is selected, simulated on the quantum architecture and stochastic weight changes seek for a decrease of the loss induced by the sample. Thus, for many training data, many iterations are needed to optimize the network weights. In contrast, we propose in this work an alternative approach, by encoding the entire training dataset as a quantum state and to sample from it:

Figure 5 shows the core idea for efficient optimization: Hadamard gates are used to encode all possible input states in superposition. The measured outcome distribution can be compared to the target distribution using e.g. a KL-divergence score: Given two probability distributions P and Q, the Kullback-Leibler-divergence (KL-divergence) or relative entropy is a dissimilarity measure for two distributions,

$$D_{KL}(P \parallel Q) = \sum_{x \in \mathcal{X}} p(x) \log \left(\frac{p(x)}{q(x)} \right) \quad (4)$$

Since the measure is differentiable, it has

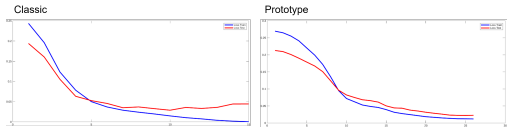


FIG. 7. Train (blue) and test (red) loss during optimization using (left) the classical perceptron model and (right) the prototype model. The prototype model shows better generalization performance.

been frequently used in the context of neural network models for a variety of downstream tasks such as image generation, noise modelling, video generation, audio generation, graph generation and more [15]. Other dissimilarity measures can be based on the cosine-divergence, optimal transport or the χ^2 measure. Especially the cosine-divergence is a useful alternative for quantum computers, as this measure is also directly expressible as quantum gate sequence [22]. The cosine-dissimilarity of two unit vectors can be expressed as its simple scalar product (denoted as $\cdot$),

$$D_{cos}(P, Q) = 1 - P \cdot Q \quad (5)$$

In our experiments we use the standard KL-divergence as optimization criteria to perform a global optimization of the network weights. Figure 6 shows the convergence behavior when using an iterative sample based optimization scheme as presented ín [26], compared to our proposed training in superposition and when using the KL-divergence. Training in superposition leads to a much faster convergence behavior.

The last experiment (bit counting) made it very easy to use Hadamard gates to represent all 2^5 possible states. In the next experiment, we use the Iris-dataset as example to train on a dataset which is not expressible with simple Hadamard-encodings. Thus, state preparation is necessary. Our workflow is as follows: First, the input data is encoded as a higher-dimensional binary vector. Taking the iris dataset as a toy example, it consists of 4 dimensional data encoding *sepal length*, *sepal width*, *petal length* and *petal width*. A kMeans clustering on each dimension with $k = 3$ is used on the train-

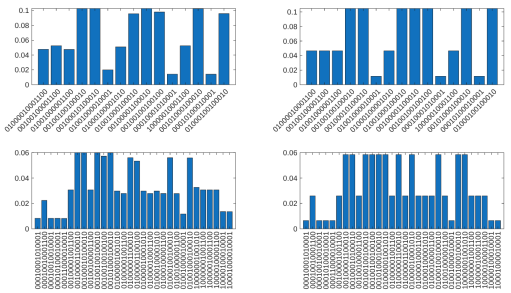


FIG. 8. Top: Training data of the iris dataset. Each bin is a data sample and the amplitude encodes the class label (three class labels are available). Bottom: Performance of the prototype model on the test dataset with amplitude encoded predicted class labels.

ing data. Thus, every datapoint can be encoded in a $4 \times 3 = 12-$dimensional binary vector which contains exactly 4 non-zero entries. Thus, a binary encoding is used to represent the dataset. Please note, that each binary encoding corresponds to a unique entry in a 2^12-dimensional vector space. Thus, 12 qubits can be brought in a superposition to represent the entire dataset. Therefore, quantum state preparation can be used, as suggested in [8, 20, 21]. It should be noted, that state preparation of an entire dataset can be a huge effort which might lead to the need of alternative approaches for optimization and representation. Besides state preparation, the term QRAM (quantum random access memory) is also commonly used in this context. Quantum Random Access Memory (QRAM) is a concept in quantum computing that generalizes the classical notion of random access memory to the quantum setting [7, 23]. A QRAM allows a quantum computer to retrieve and process large volumes of classical or quantum data while maintaining quantum coherence, such as superposition and entanglement, throughout the memory access operation. It enables the retrieval of data from any location, with multiple locations queried simultaneously via quantum superposition. Both concepts are not further investigated in this work and will be part of future research.

Our choice of preprocessing via classical clustering and binarization simplifies the classification task while encoding structure into the input. Our intention is to provide a uniform, hardware-friendly binary encoding which can also be used for other datasets while staying within the constraints of current qubit counts and connectivity. We would also like to clarify that this is just one approach to represent data with the main purpose of demonstrating our proposed concepts. For e.g. quantum signals and on-board machine learning, this needs to be adjusted accordingly.

Figure 7 shows the train and test loss during optimization using (left) the classical perceptron model and (right) the prototype model. While both models can explain the training data very well, the prototype model (with the same amount of parameters) on the right shows better generalization performance. This property of better generalization and being less prone to overfitting is one of the main reasons for the success of modern neural network architectures, e.g. based on transformers.

Figure 8 shows at the top the training data of the iris dataset. Each bin of the histogram is a dataset example and the amplitude encodes the class label. The right side shows the ground truth histogram (to be learned by the model). As there are three class labels in the dataset, the histogram shows three different height levels. The left side shows the prediction of the training dataset. Thus, the training data can be sufficiently explained by the prototype neural network. The bottom image shows on the left the predicted amplitudes using our prototype neural network model and on the right the ground truth. Even though differences are notable, when applying the regressed values to the nearest class labels, the performance of our model is in the range of 98% accuracy. This is in accordance to [26].

D. Further applications

Figure 10 shows the pipeline for training an agent to play Flappy Birds. From the image, the position of the bird, as well as the position of the tubes is translated to (input) qubit states. Based on this input, which are encoded by using RX-Gates, a quantum neural network decides, whether to fly upwards or downwards.

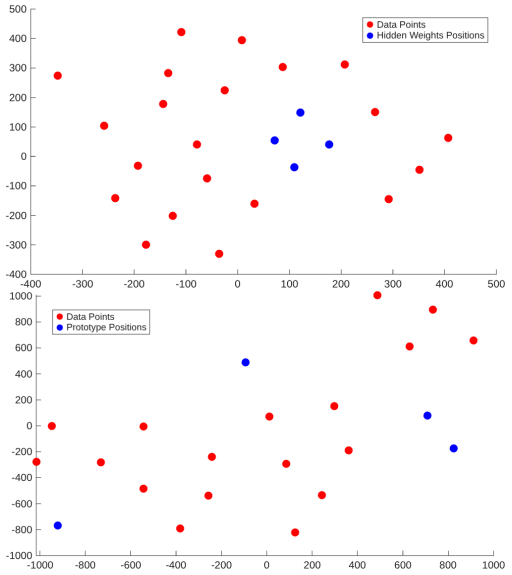


FIG. 9. Left: TSNE-plot of the training data (red) and the weights of the hidden layer (blue) when using the classical architecture. Right: TSNE-plot of the training data (red) and the weights of the prototype layer (blue) when using the prototype architecture. Whereas the hidden data reveal only limited information about the dataset, are spanning the weights of the prototype layer much better the dataset. These weights can be easier assigned to samples from the dataset and used for explanation.

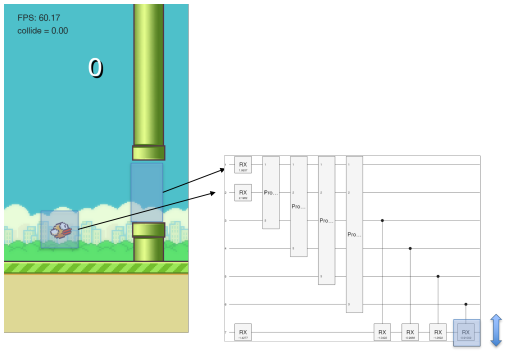


FIG. 10. Training an agent to play Flappy Birds. From the image, the position of the bird, as well as the position of the tubes is translated to qubit states. The quantum neural network decides, whether to fly upwards or downwards.

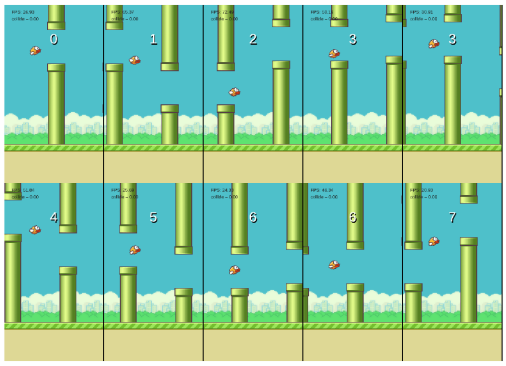


FIG. 11. Example frames of the Flappy Birds agent.

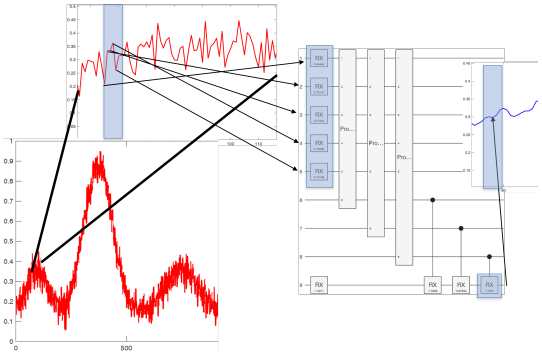


FIG. 12. Concept for sliding window based denoising an ECG-signal using a Prototype Quantum neural network. A small window of 5 samples is used as input for the neural network and the smoothed value of the middle position is estimated.

Our trained model plays the game perfectly, we stopped a run after the score of 450.000 which took 6 days. Figure 11 shows a few images of the gameplay. As model we use our prototype-architecture with 4 hidden neurons.

Motivated from future applications in direct combination of quantum sensors and AI driven quantum signal processing, we show a small use case in the final experiment: Figure 12 shows the concept for a sliding window based signal denoising task. We follow an architecture similar to Figure 10. Thus, five consecutive (noisy) samples at time point t, $[s(t-2)\dots s(t+2)]$ are used to predict the value $s(t)$. As time-series data an ECG-signal use case has been used. The goal is to denoise

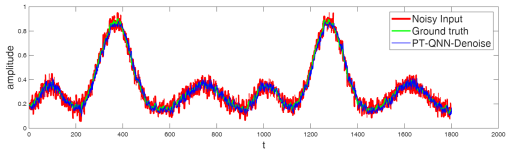


FIG. 13. Denoising an ECG-signal. The noisy input signal is shown in red, the ground truth signal is shown in green and the denoising result of our Prototype quantum neural network is shown in blue (best viewed in color and zoom in).

a signal overlaid with stochastic noise. Figure 13 shows the outcome of our model. The red curve is the noisy input signal, the ground truth signal is shown in green and the denoising result of our prototype quantum neural network is shown in blue. Table I summarizes the reconstruction quality for varying amount of prototypes. As measure we use the PSNR and the SSIM scores. The Peak Signal-to-Noise Ratio (PSNR) is defined as:

$$\mathrm{PSNR} = 10\log_{10}\left(\frac{R^2}{\mathrm{MSE}(s_1,s_2)}\right) \tag{6}$$

$$= 20\log_{10}\left(\frac{R}{\sqrt{\mathrm{MSE}(s_1,s_2)}}\right) \tag{7}$$

with MSE denoting the Mean Squared Error and R denoting the maximum fluctuation in the signal. The Structural Similarity (SSIM) index is defined as

$$\mathrm{SSIM}(s_1,s_2) = \frac{(2\mu_{s_1}\mu_{s_2}+C_1)(2\sigma_{s_1s_2}+C_2)}{(\mu_{s_1}^2+\mu_{s_1}^2+C_1)(\sigma_{s_1}^2+\sigma_{s_1}^2+C_2)} \tag{8}$$

where μ and σ denote the mean, variance and covariance of the signals and C are (small) stabilizer constants to avoid division by zero. Table I shows that already a few prototypes are sufficient to denoise the ECG-signal significantly. Our model outer-performs the classical perceptron, but it should be noted that both models solve the task well.

	CQNN		PTQNN (ours)	
#LS,PT	PSNR ↑	SSIM ↑	PSNR ↑	SSIM ↑
1	32.72	0.90	32.9	0.87
2	38.20	0.90	40.00	0.93
3	39.65	0.92	40.04	0.93

TABLE I. Evaluation of reconstruction quality for noisy ECG signals for different amount of prototype or latent space layers. CQNN denotes the classical quantum neural network with 1-3 neurons as latent space dimension in the hidden layer and PTQNN denotes the prototype model with 1-3 prototype layers. Measured are the Peak signal to noise ration (PSNR) and the structural similarity index (SSIM). Already a few prototypes or neurons are sufficient to denoise the ECG-signal significantly.

IV. SUMMARY AND OUTLOOK

In this work, we introduced a quantum prototype perceptron architecture that extends classical quantum neural network models towards prototype-based learning. Additionally we present efficient, distribution-level optimization utilizing quantum superposition. In contrast to conventional perceptron approaches that rely solely on weighted summation, it is well known that such models are susceptible to overfitting and limited interpretability. Instead, our method leverages learnable prototypes and XOR-type feature comparisons to enable both zero- and one-valued input features to actively contribute to neuron activation. This bidirectional sensitivity results in more expressive and interpretable representations, aligning conceptually with mechanisms underlying modern attention architectures. The second main contribution of this work lies in the proposed quantum optimization strategy, which encodes the complete training dataset as a superposed quantum state, thus permitting simultaneous, global parameter updates by directly minimizing the divergence between target and model distributions. Our empirical studies across benchmark datasets demonstrate that the quantum prototype model achieves superior stability and generalization, and is notably robust to overfitting compared to clas-

sical and other quantum perceptron models. Applications towards simple game play or time series analysis demonstrate the general possibilities of the proposed concept.

Integrating our method with recent advances in quantum state preparation and error mitigation could further enhance practical applicability of our method. The inherent interpretability of the prototype framework opens new opportunities for explainable quantum AI, which may support trustworthy quantum-assisted decision-making in scientific and technological domains.

Funding Declaration

This work was supported, in part, by the Federal Ministry of Research, Technology and Space (BMFTR), Germany under the AI service center KISSKI (grant no. 01IS22093C), the QC service center QUICS (grant no. 13N17418), by the Quantum Valley Lower Saxony and by Germany's Excellence Strategies EXC-2122 PhoenixD and EXC-2123 Quantum Frontiers. We would like to thank Tobias Osborne from the Institute for Theoretical Physics (ITP) at the Leibniz University of Hannover, Germany for fruitful discussions and hints in this project.

[1] Johannes Bausch. Recurrent quantum neural networks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc.

[2] Kerstin Beer, Dmytro Bondarenko, Terry Farrelly, Tobias J. Osborne, Robert Salzmann, Daniel Scheiermann, and Ramona Wolf. Training deep quantum neural networks. *Nature Communications*, 11(1):808, Feb 2020.

[3] Pablo Bermejo, Paolo Braccia, Manuel S. Rudolph, Zoë Holmes, Lukasz Cincio, and M. Cerezo. Quantum convolutional neural networks are (effectively) classically simulable, 2024.

[4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc.

[5] Yudong Cao, Gian Giacomo Guerreschi, and Alán Aspuru-Guzik. Quantum neuron: an elementary building block for machine learning on quantum computers, 2017.

[6] Thomas Cover and Peter Hart. *Nearest neighbor pattern classification*, volume 13. IEEE, 1967.

[7] Vittorio Giovannetti, Seth Lloyd, and Lorenzo Maccone. Quantum random access memory. *Phys. Rev. Lett.*, 100:160501, Apr 2008.

[8] Lov K. Grover. Synthesis of quantum superpositions by quantum computation. *Phys. Rev. Lett.*, 85:1334–1337, Aug 2000.

[9] Barbara Hammer and Thomas Villmann. Generalized relevance learning vector quantization. *Neural Netw.*, 15(8–9):1059–1068, October 2002.

[10] Noah Hollmann, Samuel Müller, Lennart Purucker, Arjun Krishnakumar, Max Körfer, Shi Bin Hoo, Robin Tibor Schirrmeister, and Frank Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637(8045):319–326, Jan 2025.

[11] Subhash C. Kak. Quantum neural computing. volume 94 of *Advances in Imaging and Electron Physics*, pages 259–313. Elsevier, 1995.

[12] J. Kiefer and J. Wolfowitz. Stochastic Estimation of the Maximum of a Regression Function. *The Annals of Mathematical Statistics*, 23(3):462 – 466, 1952.

[13] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollar, and Ross Girshick. Segment anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4015–4026, October 2023.

[14] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[15] Ivan Kobyzev, Simon Prince, and Marcus Brubaker. Normalizing flows: An introduction and review of current methods. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PP:1–1, 05 2020.

[16] Lasse Bjørn Kristensen, Matthias Degroote, Peter Wittek, Alán Aspuru-Guzik, and Nikolaj T. Zinner. An artificial spiking quantum neuron. *npj Quantum Information*, 7(1):59, Apr 2021.

[17] Junwei Ma, Valentin Thomas, Rasa Hosseinzadeh, Hamidreza Kamkari, Alex Labach, Jesse C Cresswell, Keyvan Golestan, Guangwei Yu, Anthony L Caterini, and Maksims Volkovs. Tabdpt: Scaling tabular foundation models on real data. *arXiv preprint arXiv:2410.18164*, 2024.

[18] Gengchen Mai, Weiming Huang, Jin Sun, Suhang Song, Deepak Mishra, Ninghao Liu, Song Gao, Tianming Liu, Gao Cong, Yingjie Hu, Chris Cundy, Ziyuan Li, Rui Zhu, and Ni Lao. On the opportunities and challenges of foundation models for geoai (vision paper). *ACM Trans. Spatial Algorithms Syst.*, 10(2), July 2024.

[19] Christopher D. Manning. Human language understanding & reasoning. *Daedalus*, 151(2):127–138, 05 2022.

[20] Sam McArdle, András Gilyén, and Mario Berta. Quantum state preparation without coherent arithmetic. *Phys. Rev. Lett.*, 136:240603, Jun 2026.

[21] Mikko Möttönen, Juha J. Vartiainen, Ville Bergholm, and Martti M. Salomaa. Quantum circuits for general multiqubit gates. *Phys. Rev. Lett.*, 93:130502, Sep 2004.

[22] D. Pastorello and E. Blanzieri. A quantum binary classifier based on cosine similarity. In *2021 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 477–478, Los Alamitos, CA, USA, oct 2021. IEEE Computer Society.

[23] Koustubh Phalak, Avimita Chatterjee, and Swaroop Ghosh. Quantum random access memory for dummies. *Sensors (Basel)*, 23(17):7462, August 2023.

[24] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.

[25] F. Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408, 1958.

[26] Bodo Rosenhahn, Tobias J. Osborne, and Christoph Hirche. Stochastic neural networks for quantum devices, 2026.

[27] Atsushi Sato and Keiji Yamada. Generalized learning vector quantization. In *Proceedings of the 9th International Conference on Neural Information Processing Systems*, NIPS'95, page 423–429, Cambridge, MA, USA, 1995. MIT Press.

[28] Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical networks for few-shot learning. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

[29] Jake Snell, Kevin Swersky, and Richard S Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems (NIPS)*, volume 30, 2017.

[30] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc.

[31] Jingyi Zhang, Jiaxing Huang, Sheng Jin, and Shijian Lu. Vision-language models for vision tasks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(8):5625–5644, 2024.